

# Lu Delivery – Sistema de Pedidos para Restaurantes com Programação Orientada a Objetos

Erica A. de Jesus, Nalbert S. Santana, Mickael C. Alencar, Monyc L. A. Cerqueira, Pedro César. P. Jesus

Sistemas de Informação - Centro Universitário de Excelência (UNEX) Av. Artêmia Pires Freitas, s/n - Sim - 44085-370 - Feira de Santana - BA – Brasil

erica.jesus2@aluno.unex.edu.br, nalbert.santana@aluno.unex.edu.br,  
mickael.alencar monyc.cerqueira@aluno.unex.edu.br,  
cesar.jesus2@aluno.unex.edu.br Introdução

**Abstract.** *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

**Resumo.** *Este trabalho apresenta o desenvolvimento e refatoração do sistema FoodDelivery, implementado em Java, para o gerenciamento de pedidos em restaurante. O sistema aplica conceitos de programação orientada a objetos, organizando de forma clara as entidades e processos do mundo real. A implementação do padrão Builder Encadeado garantiu que cliente e itens fossem definidos antes da finalização do pedido, evitando estados inválidos. Operando por meio de uma interface de linha de comando (CLI), o sistema permite gerenciar cardápio, clientes, pedidos e emitir relatórios simplificados. Os resultados demonstram a eficácia da abordagem orientada a objetos e do uso de padrões de projeto na criação de sistemas mais seguros e de fácil manutenção.*

## 1. Introdução

A Programação Orientada a Objetos (POO) é um paradigma fundamental para o desenvolvimento de sistemas de software complexos e de longa manutenção. Ao centralizar o design em entidades (objetos) que encapsulam estado e comportamento, a POO com seus pilares de abstração, encapsulamento, herança e polimorfismo, proporciona a flexibilidade e a modularidade necessárias para a escalabilidade de aplicações como a FoodDelivery.

A FoodDelivery é uma aplicação de pedidos de comida que tem como propósito conectar clientes e restaurantes em uma plataforma digital, permitindo o gerenciamento de cardápios, pedidos e entregas de forma eficiente e modular. Com a expansão da aplicação e o aumento das funcionalidades, surgiram desafios de complexidade e manutenção no código, especialmente no processo de criação de pedidos, cuja lógica se tornou desorganizada e suscetível a erros.

Diante desse problema, o objetivo principal deste trabalho foi refatorar o processo de criação de pedidos, aplicando o padrão de projeto Builder, que permite separar a lógica de construção de um objeto da sua representação final. A adoção desse padrão visa melhorar a legibilidade do código, reduzir falhas de inicialização fora de ordem e aumentar a segurança e extensibilidade do sistema.

Como resultado, obteve-se um código mais limpo, seguro e modular, favorecendo a evolução sustentável da aplicação. Nas próximas seções, este relatório apresenta a fundamentação teórica sobre POO e padrões de projeto, descreve detalhadamente a metodologia e implementação do padrão Builder no contexto da FoodDelivery, e, por fim, discute os resultados, desafios e aprendizados obtidos durante o desenvolvimento.

## 2. Fundamentação Teórica

A consolidação da Programação Orientada a Objetos (POO) exige compreensão profunda de seus princípios estruturantes e práticas de design associadas (LARMAN, 2004). Nesta seção, são apresentados os principais conceitos que fundamentam a refatoração realizada, relacionando-os ao padrão de projeto Builder, aplicado na construção controlada de objetos complexos.

### 2.1 Pilares da POO

A Herança possibilita que as classes compartilhem seus atributos, métodos e outros membros da classe entre si. Para a ligação entre as classes, a herança adota um relacionamento esquematizado hierarquicamente. Na Herança temos dois tipos principais de classe:

1. **Classe Base:** A classe que concede as características a uma outra classe.
2. **Classe Derivada:** A classe que herda as características da classe base.

O fato de as classes derivadas herdarem atributos das classes bases assegura que programas orientados a objetos cresçam de forma linear e não geometricamente em complexidade. Cada nova classe derivada não possui interações imprevisíveis em relação ao restante do código do sistema.

Definimos Polimorfismo como um princípio a partir do qual as classes derivadas de uma única classe base são capazes de invocar os métodos que, embora apresentem a mesma assinatura, comportam-se de maneira diferente para cada uma das classes derivadas. O Polimorfismo é um mecanismo por meio do qual selecionamos as funcionalidades utilizadas de forma dinâmica por um programa no decorrer de sua execução. Com o Polimorfismo, os mesmos atributos e objetos podem ser utilizados em objetos distintos, porém, com implementações lógicas diferentes. Por exemplo: podemos assumir que uma

bola de futebol e uma camisa da seleção brasileira são artigos esportivos, mais que o cálculo deles em uma venda é calculado de formas diferentes.

## 2.2 Padrões de Projeto

Padrões de projeto descrevem soluções testadas para problemas recorrentes em arquitetura de software, padronizando interações entre classes e objetos (GAMMA et al., 1994). Eles podem ser classificados em:

1. **Criacionais:** Focados na criação de objetos de maneira controlada, aumentando a flexibilidade do sistema.
2. **Estruturais:** Definem como classes e objetos se combinam para formar estruturas maiores.
3. **Comportamentais:** Descrevem como objetos interagem e distribuem responsabilidades.

O uso desses padrões reduz a duplicação de código, melhora a manutenibilidade e padroniza a comunicação entre desenvolvedores. No contexto deste trabalho, o padrão Builder foi escolhido para solucionar o problema da construção desordenada de pedidos, garantindo a integridade do objeto final.

## 2.3 Padrão Builder (Construtor)

O padrão Builder é um padrão criacional cujo objetivo principal é separar o processo de construção de um objeto complexo de sua representação final, permitindo que o mesmo processo produza diferentes representações.

De forma geral, o Builder é empregado quando um objeto exige múltiplos passos obrigatórios de inicialização, tornando o construtor tradicional insuficiente. Ele atua como um intermediário que controla a sequência de montagem do objeto, garantindo que ele só seja retornado ao usuário quando estiver completo e válido.

No contexto da aplicação FoodDelivery, o problema identificado foi a criação desorganizada de objetos Pedido, permitindo que fossem instanciados fora de ordem, por exemplo, antes da definição de um cliente ou sem itens no carrinho. O OrderBuilder foi então projetado para eliminar essas inconsistências, garantindo uma sequência lógica e obrigatória de etapas:

```
Order minhaOrder = OrderBuilder.newOrder()
```

```
.setCustomer(meuCustomer)
```

```
.addItem(menuItem1, 2)
```

```
.addItem(menuItem2, 1)
```

```
.finishItems()
```

```
.build();
```

Com isso, o código se torna mais legível, seguro e modular, pois a construção do objeto Order ocorre por meio de métodos encadeados que seguem uma sequência predefinida. Essa solução está alinhada à visão de Larman (2004), segundo a qual “um bom design orientado a objetos deve enfatizar baixo acoplamento, alta coesão e clareza nas dependências entre classes.”

### **3. Metodologia**

A refatoração da lógica de criação de pedidos focou na aplicação do Padrão Builder Encadeado (Guiding Builder) para garantir que o processo de construção segue uma ordem: (1) Definir o Cliente, (2) Adicionar Itens, (3) Finalizar a adição de Itens e (4) Construir o Pedido.

#### **3.1 Implementação do Padrão**

O processo de implementação baseou-se na definição de interfaces aninhadas, representando cada estágio obrigatório da construção do objeto. Cada interface define o método que a conclui e retorna o tipo correspondente à próxima etapa, estabelecendo uma ordem obrigatória de execução.

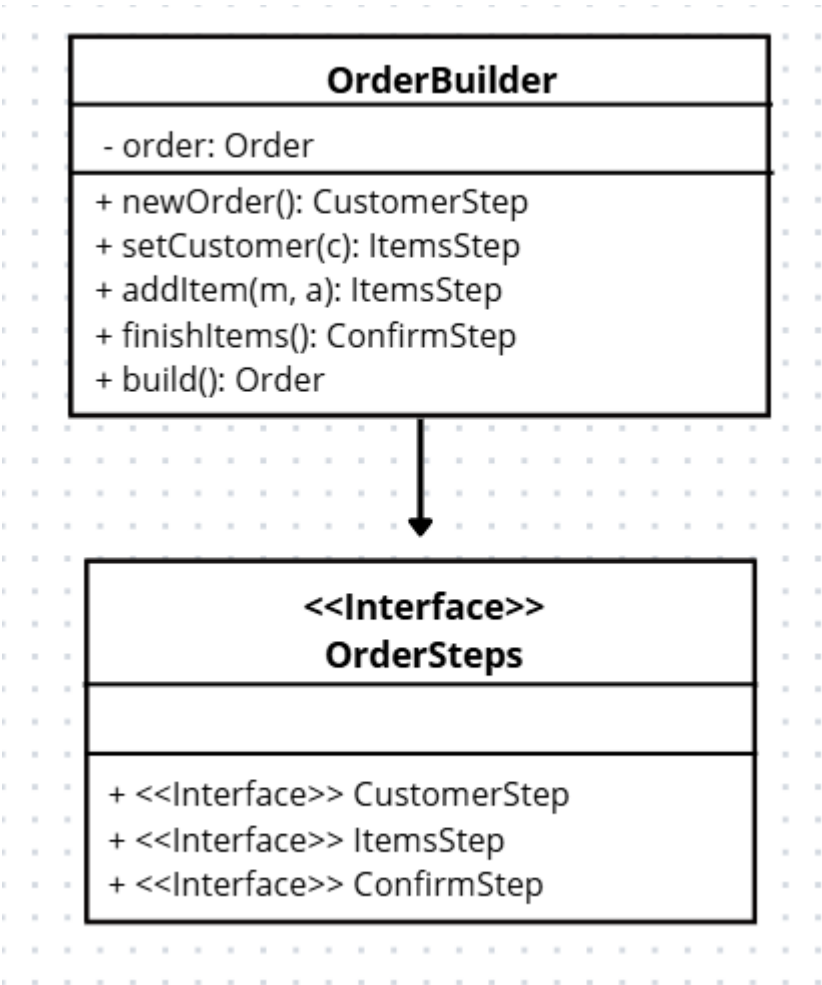
A classe concreta OrderBuilder foi responsável por implementar todas as interfaces, centralizando a lógica de construção e gerenciando o estado interno. Cada método foi projetado para retornar apenas o tipo da próxima etapa lógica — por exemplo, setCustomer() retorna ItemsStep, restringindo o acesso a métodos fora da sequência esperada.

Por fim, o objeto Order é instanciado apenas dentro do método build(), definido na última interface (ConfirmStep). Esse método marca o término do processo de construção e encapsula todas as etapas anteriores.

4. Resultados e Discussões

Figura 1. Diagrama de Classes

Essa modelagem garante que a classe OrderBuilder implemente as interfaces definidas em OrderSteps, assegurando que cada etapa leve obrigatoriamente à próxima — eliminando a possibilidade de saltos indevidos na sequência de construção.



| Componente   | Tipo                | Responsabilidade                                                                                                                                     |
|--------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| OrderBuilder | Classe Concreta     | Implementa as interfaces de etapas e gerencia o estado interno da construção do Pedido. Contém o método estático newOrder() para iniciar o processo. |
| OrderSteps   | Interface Principal | Define as interfaces aninhadas (CustomerStep, ItemsStep, ConfirmStep) que representam a ordem sequencial e obrigatória dos passos de construção.     |
| Order        | Classe Produto      | O objeto complexo a ser construído.                                                                                                                  |

5. Considerações Finais

A refatoração do sistema FoodDelivery com o padrão Builder Encadeado apresentou desafios na modelagem das interfaces para garantir a sequência correta de construção dos pedidos. Foi interessante perceber como o Builder torna o fluxo claro e seguro, prevenindo erros de inicialização e aumentando a legibilidade do código. Com mais

tempo, o foco seria aprimorar a implementação básica, garantindo que cada etapa do Builder funcionasse de forma completamente confiável antes de considerar funcionalidades adicionais. A experiência demonstrou que o padrão Builder é eficaz para organizar a criação de objetos complexos, tornando o sistema mais organizado e de fácil manutenção.

## **Referências**

**Sommerville, I.** *Software Engineering*, 9th edition, Addison-Wesley, England, 2011.

**DevMedia.** “Conceitos e exemplos: Polimorfismo na Programação Orientada a Objetos”, Disponível em <https://www.devmedia.com.br/conceitos-e-exemplos-polimorfismo-programacao-orientada-a-objetos/18701>, Accessed October 2025.

**Trasel, R.** “Usando Padrão de Projeto Builder com Java”, <https://medium.com/@robson.trasel/usando-padr%C3%A3o-de-projeto-builder-com-java-6b7001310e6d>, Accessed October 2025.

**ORACLE.** Java Platform, Standard Edition Documentation. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 28 ago. 2025.